

PROJECT: CROWD MANAGEMENT SIMULATION (EXIT SELECTION)

Final Project Report

Done by:

Wasan Sulaiman AL-Nabhani

ID: 20-0243

Instructor:

Dr. Lilia Rejeb

22/05/2026

1. Introduction

The project develops a 2D crowd evacuation simulation system where pedestrians can choose their exits according to the trained tabular Q-learning policy. Not only can pedestrians get to their exit point, but also they can make their choice while selecting their exits depending on how close and congested they are.

This project will be compared to two other systems as its benchmark, and they include choosing the closest exit and choosing an exit randomly.

Moreover, this project incorporates the use of agent-based AI technology. The local LLM narrator will view the simulated environment through the results obtained from tools used. The narrator will narrate one sentence every 30 timesteps. At the end of each episode, three paragraphs of the chronological story will be written.

2. System Architecture

There are three levels to the implementation, namely, the simulator level, the reinforcement learning level, and the LLM narrator level. The simulator generates positions, exit loads, and density statistics. The Q-learning policy then selects exits based on the state variables generated by the simulator. The narrator makes use of the tool level to convert simulator statistics into explanations.

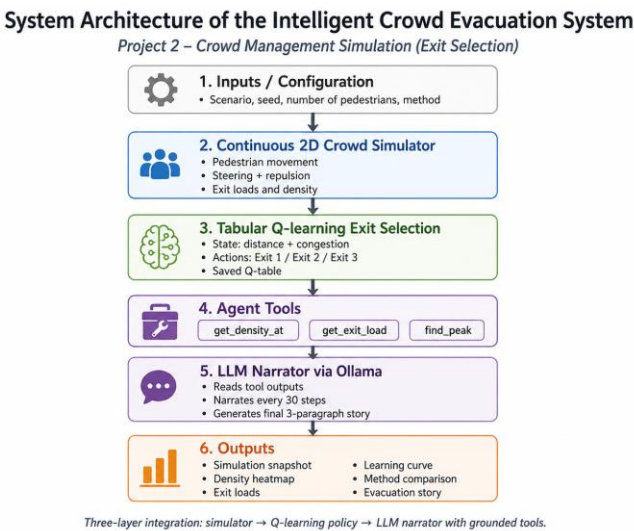


Figure 1. Three-layer architecture of the crowd evacuation system

3. Core Simulator Design

The main technical component is a 2D continuous evacuation simulator. The room dimensions are 100 by 100 units, and there are three exits available in the room. The pedestrians are modeled using agents with properties such as position, velocity, selected exit, and evacuation

status. The movement strategy is similar to the boid model where the pedestrian moves towards its selected exit but avoids other pedestrians to decrease overlapping.

In order to increase congestion, there is a limitation on the capacity of each exit per step, which leads to a bottleneck effect. This means that being closer to an exit does not necessarily imply being a good choice of action. There are two possible scenarios for spawning pedestrians in the simulator uniform and corner-heavy.

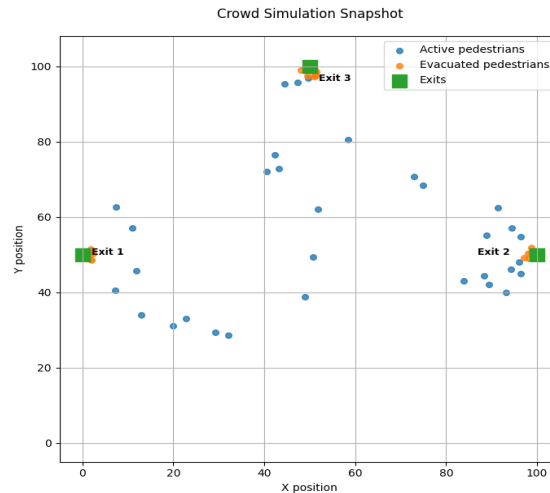


Figure 2. Simulator snapshot showing active pedestrians, evacuated pedestrians, and exits.

Heatmap of Density is derived based on the position of the pedestrians during evacuation. This indicates areas with local density of crowd. The heat map is used as a debugging aid and helps verify if the tool layer contains density information that can be given to the narrator.

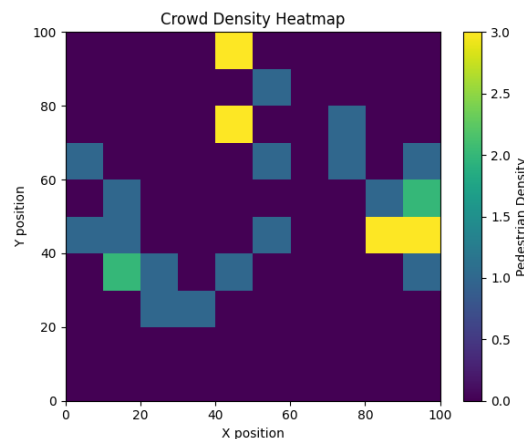


Figure 3. Density heatmap showing local crowd concentration during the evacuation.

4. MDP Formulation and Q-learning Design

The reinforcement learning component is formulated as a single-agent exit-choice MDP. The action is not a movement command; instead, the policy selects which exit a pedestrian should head toward. The continuous simulator then handles the motion using steering and repulsion.

MDP Element	Project Implementation
State	Discretized distance to each exit + discretized density/load at each exit
Actions	0 = Exit 1, 1 = Exit 2, 2 = Exit 3
Reward	Lower cost for shorter distance and lower congestion; bonus for selecting lowest-cost exit
Policy	Tabular Q-learning with epsilon-greedy exploration
Deployment	The saved Q-table is used by all pedestrians in the multi-agent simulator

With Q-learning being able to use only discrete states, both continuous distances and exit loads were discretized into categories such as "near", "medium", "far" and "low", "medium", "high". The state, when there are three exits, consists of both distances and exit loads for all of the exits. While training, the system utilized random pedestrian positions and stochastic exit loads. Following training, the Q-table that was saved was used by the simulator to determine the most optimal exit.

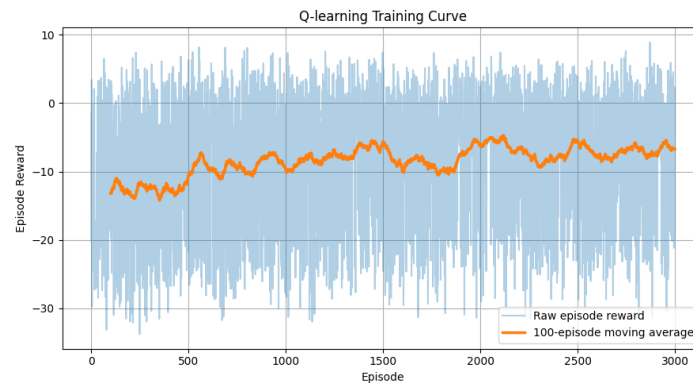


Figure 4. Q-learning training curve. The orange line is a 100-episode moving average that makes the learning trend clearer.

5. Agentic AI Layer

The agent AI module is realized as a one-loop narrator. It doesn't have any control over the pedestrians. Rather, it takes the grounded tool outputs produced by the simulator and converts them into brief factual descriptions of the evacuation process.

The project is based on a local Ollama model, llama3.2:3b, along with a fallback text generator in case the local model fails to work.

Tool	Purpose
<code>get_density_at(x, y)</code>	Returns the number of active pedestrians near a selected location
<code>get_exit_load(exit_id)</code>	Returns current load at one exit or all exits
<code>find_peak()</code>	Finds the highest-density area in the room using the heatmap

After each 30 steps of simulation, the narrator gets the current tool description and generates precisely one sentence. In the end, using the timeline and final data, it creates a three-paragraph story consisting of an introduction, the behavior during congestion, and the final result.

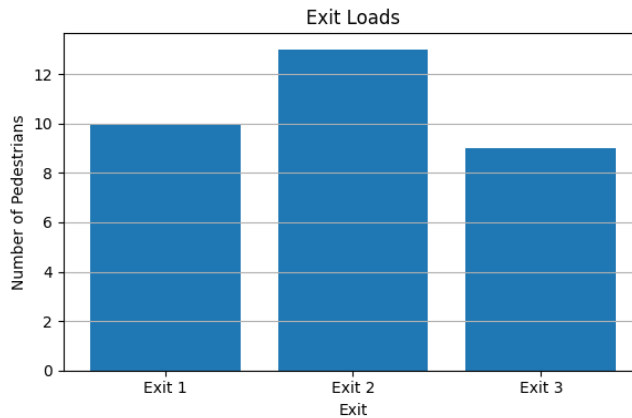


Figure 5. Exit-load distribution during evacuation. Exit 2 has the highest temporary load in this run

Example runtime narration: At step 30, the evacuation continued with exit loads {Exit 1: 10, Exit 2: 13, Exit 3: 9} and a peak local density of 3.

6. Prompt Iteration

Prompt iterations for five times were recorded in docs/prompt_iterations. The prompt was improved from a generic prompt asking for a narration to a grounded prompt which specifically instructs the model to make use of only the supplied metrics, not make assumptions without any basis, generate a single sentence at runtime, and construct a three-paragraph story.

7. Evaluation Results

The experiment evaluates three different approaches: the Q-learning policy after training, the nearest-exit approach, and the random approach. The experiment consists of two cases, three seeds, and 50 pedestrians to form a challenging scenario. The key performance indicators include average evacuation time, completion rate, and simulation steps.

Method	Avg. Evacuation Time	Completion Rate	Avg. Total Steps
nearest_exit	37.97	1.00	63.33
q_learning	48.26	1.00	110.00
random_choice	148.63	0.97	500.00

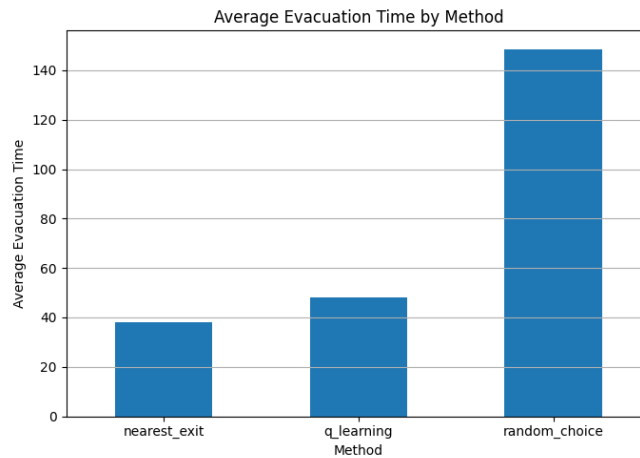


Figure 6. Average evacuation time comparison across methods

It is clear that the Q-learning approach performs much better than the random choice approach. It achieves perfect completeness and low average evacuation times compared to the random choice approach. The nearest exit algorithm is still the fastest since distance is a primary consideration and there are no obstacles within the simple room structure.

This is a true limitation of the simulator and not of the Q-learning algorithm itself.

8. Demonstration Run

Metric	Value
Scenario	uniform
Pedestrians	50
Total steps	123
Evacuated count	50
Completion rate	1.00
Average evacuation time	47.62 steps

9. Testing and Reproducibility

The project features a pytest suite for the simulator, density heatmap calculation, Q-learning policy, agent utilities, and three-layer integration. The last set of tests executed resulted in 13 tests, all of which passed successfully.

Area Tested	What Was Checked
Simulator	Pedestrians are created, move, and return valid metrics
Density	Heatmap shape and active-pedestrian counts are valid
Q-learning	Actions are valid and Q-values update
Agent tools	Density, exit-load, and peak tools return valid outputs
Integration	Simulator, Q-learning policy interface, tools, and narrator run together

Final test result: 13 passed. One-command run: `python main.py --mode full --method q_learning --scenario uniform --n_pedestrians 50 --seed 1`

10. Limitations

The simulator is deliberately made simple. It does not account for panic, injuries, complicated psychology, complicated geometry of buildings, varied walking speed, and obstacles. The Q-learning state space is discretized, resulting in loss of some information. The narrator is based on measurements; however, it does not affect the simulation and the pedestrian movement in any way.

11. AI Assistance Provenance and Individual Contribution

This project was completed independently by me, Wasan Sulaiman AL-Nabhani (ID 20-0243). AI technology has been employed to assist in coding, debugging, prompt improvements, test case development, enhancement of documentation, and developing the simplified diagram for the system architecture. All codes, results, diagrams, simulations, graphs, and test cases have been checked, verified, and improved by me.

12. Conclusion

The project incorporates all the three layers in the following manner: continuous crowd simulator, tabular Q-learning exit selection strategy, and a narrator that uses tools and is a local large language model. The system works from end-to-end, saves a Q-table, creates plots, makes visualizations of crowd density and exit loads, evaluates against baselines, and tells the story.